

WhatNext Vision Motors: Shaping the Future of Mobility with Innovation and Excellence

Use Case :

WhatsNext Vision Motors, a pioneering force in the automotive industry, is dedicated to transforming the mobility sector with innovative technology and solutions that prioritize customer needs. The company has embarked on an ambitious Salesforce project with the core objective of enhancing the customer experience and streamlining its operational processes.

At the heart of this project is the improvement of the customer ordering process. The system is designed to automatically suggest the nearest dealer location to customers based on their address. This feature is intended to significantly enhance the convenience and efficiency of the ordering experience, making it more customer-friendly and reducing the time and effort required from the customer's end.

The project also addresses a common issue in the automotive industry: stock availability. The system includes a mechanism that prevents customers from placing orders for vehicles that are out of stock. This proactive approach ensures that customers can only create orders for vehicles that are currently available, thus avoiding potential confusion and disappointment that may arise from stock unavailability. This feature not only enhances customer satisfaction but also improves the accuracy of the company's order fulfillment process.

Furthermore, the project incorporates a scheduled process for updating the status of bulk order records. This automated process is designed to update the order status based on stock availability. If a vehicle is out of stock at the time of order placement, the system will update the order status to 'Pending.' On the other hand, if the vehicle is in stock, the system will update the status to 'Confirmed.' This ensures that all orders are accurately reflected in terms of their fulfillment status, providing clear and transparent communication to customers regarding the status of their orders.

The implementation of this Salesforce project at WhatsNext Vision Motors is expected to yield several benefits. It aims to create a more efficient ordering system that reduces the potential for errors and improves the overall service provided to customers. By streamlining the ordering process and ensuring accurate stock availability, the company can enhance customer satisfaction and loyalty.

Moreover, the project is expected to contribute to operational efficiency by reducing the administrative burden on staff. By automating certain processes, employees can focus on more strategic tasks that require human intervention and expertise. This not only improves the overall

productivity of the company but also allows for a more agile response to market demands and customer needs.

Requirements :

1. Salesforce CRM Implementation

- Store and manage vehicle details, stock availability, and dealer information in Salesforce.
- Track customer orders, test drives, and service requests efficiently.
- Automate workflows to assign orders to the nearest dealer based on customer location.

2. Process Automation

- Prevent order placement if the vehicle is out of stock.
- Auto-assign orders to the nearest dealer based on the customer's location.
- Send automated email reminders for scheduled test drives.

3. Apex and Trigger

- Implement Apex triggers to enforce business rules such as stock validation and automatic dealer assignment.
- Use trigger handlers to follow best practices and ensure modularity and maintainability.

4. Batch Jobs

- Develop a batch Apex job to periodically check vehicle stock levels and update availability.
- Send scheduled email notifications for stock replenishment and order processing.

What you'll learn :

1. Data Modelling
2. Fields and Relationships
3. Lightning App Builder
4. Record Triggered Flows
5. Apex and Apex Triggers
6. Batch Apex
7. Scheduled Apex

MileStone 1 :

Activity 1: Creating Developer Account

Creating a developer org in salesforce.

1. Go to <https://developer.salesforce.com/signup>
2. On the sign up form, enter the following details :

- 1) First name & Last name
- 2) Email
- 3) Role : Developer
- 4) Company : College Name
- 5) Country : India
- 6) Postal Code : pin code
- 7) Username : should be a combination of your name and company

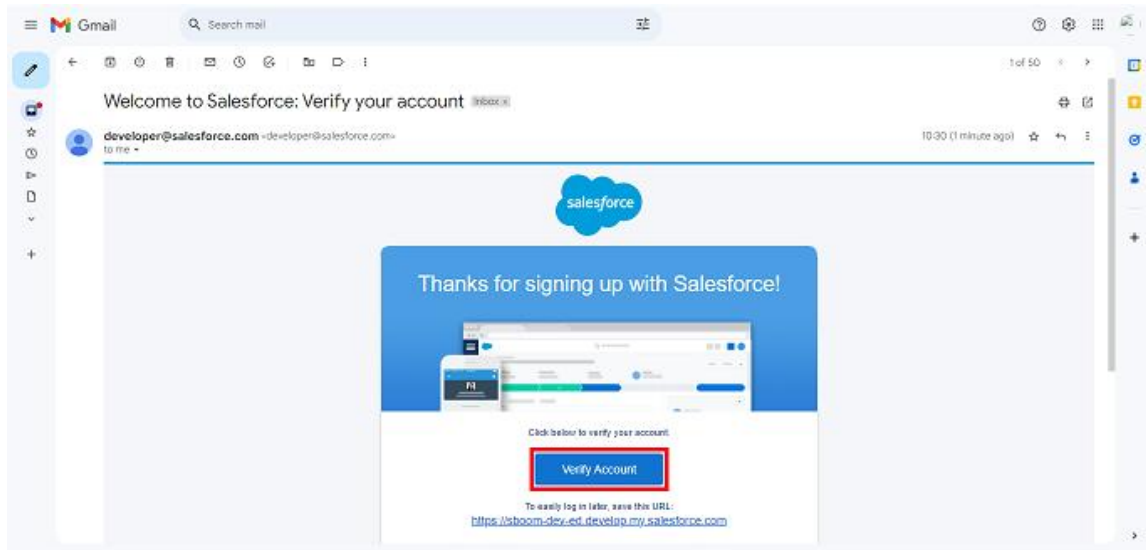
This need not be an actual email id, you can give anything in the format :

username@organization.com

Click on sign me up after filling these.

Activity 2: Account Activation

1. Go to the inbox of the email that you used while signing up. Click on the verify account to activate your account. The email may take 5-10mins.



2. Click on Verify Account
3. Give a password and answer a security question and click on change password.

Change Your Password

Enter a new password for lead@sb.com.
Make sure to include at least:

- ✓ 8 characters
- ✓ 1 letter
- ✓ 1 number

* New Password
Good

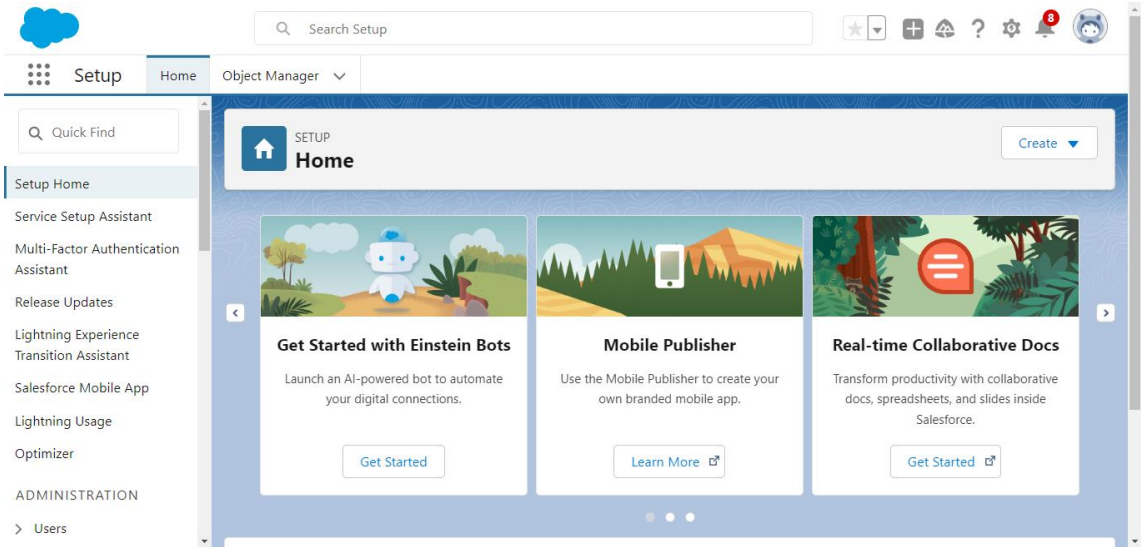
* Confirm New Password
Match

Security Question
In what city were you born?

* Answer
asdfghjkl

Change Password

4. Then you will redirect to your salesforce setup page.



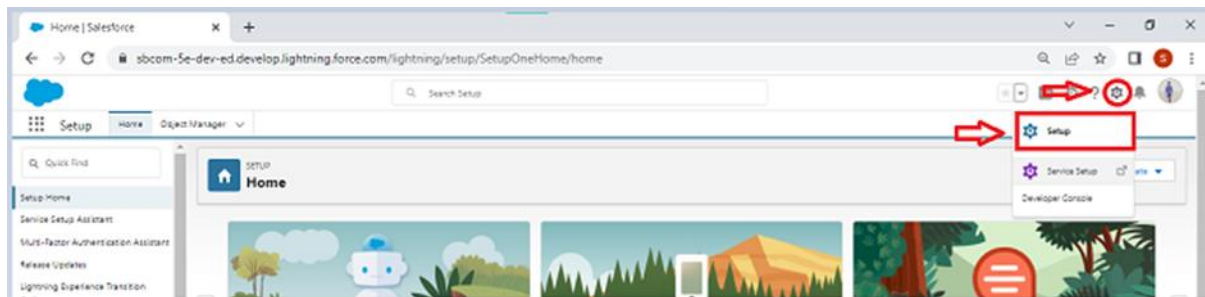
MileStone 2 :

Objects & Relationships

Object Name	Purpose	Relationships
Vehicle__c	Stores vehicle details	Related to Dealer & Orders
Vehicle_Dealer__c	Stores authorized dealer info	Related to Orders
Vehicle_Customer__c	Stores customer details	Related to Orders & Test Drives
Vehicle_Order__c	Tracks vehicle purchases	Related to Customer & Vehicle
Vehicle_Test_Drive__c	Tracks test drive bookings	Related to Customer & Vehicle
Vehicle_Service_Request__c	Tracks vehicle servicing requests	Related to Customer & Vehicle

To Navigate to Setup page:

Click on gear icon → click setup.



Activity 1: Create Vehicle Object

The purpose of creating a Vehicle custom object is to have a clear picture of the vehicle details.

To create an object:

1. From the setup page → Click on Object Manager → Click on Create → Click on Custom Object.



- 1) Enter the label name→ Vehicle
- 2) Plural label name→ Vehicles
- 3) Enter Record Name Label and Format
 - Record Name → Vehicle Name
 - Data Type → Text

SETUP
New Custom Object

Custom Object Definition Edit Save Save & New Cancel

Custom Object Information
The singular and plural labels are used in tabs, page layouts, and reports.

Label Example: Account
 Plural Label Example: Accounts
☐ Starts with vowel sound

The Object Name is used when referencing the object via the API.

Object Name Example: Account
 Description

Context-Sensitive Help Setting
☒ Open the standard Salesforce.com Help & Training window
☐ Open a window using a Visualforce page

Content Name

Enter Record Name Label and Format
The Record Name appears in page layouts, key lists, related lists, lookups, and search results. For example, the Record Name for Account is "Account Name" and for Case it is "Case Number". Note that the Record Name field is required.

Record Name Example: Account Name
 Data Type **Warning:** If you plan to insert a high volume of records in this object, via the API for example, use the Text data type.

2. Click on Allow reports,
3. Allow search → **Save.**

The screenshot shows the 'Optional Features' section of the Salesforce Custom Object Setup page. Red arrows point to the following elements:

- Allow Reports:** A red arrow points to the checked checkbox.
- Allow Search:** A red arrow points to the checked checkbox.
- Save Buttons:** A red arrow points down to the 'Save' button, and another red arrow points up to the 'Save & Next' button.

Other visible settings include:

- Object Classification:** 'Allow Sharing', 'Allow Bulk API Access', and 'Allow Streaming API Access' are all checked.
- Deployment Status:** 'Deployed' is selected.
- Object Creation Options:** 'Add Notes and Attachments related list to default page layout' and 'Launch New Custom Tab Wizard after saving this custom object' are unchecked.

Activity 2: Create Dealer Object

The purpose of creating a Dealer object is to have detailed information about Dealer.

To create an object:

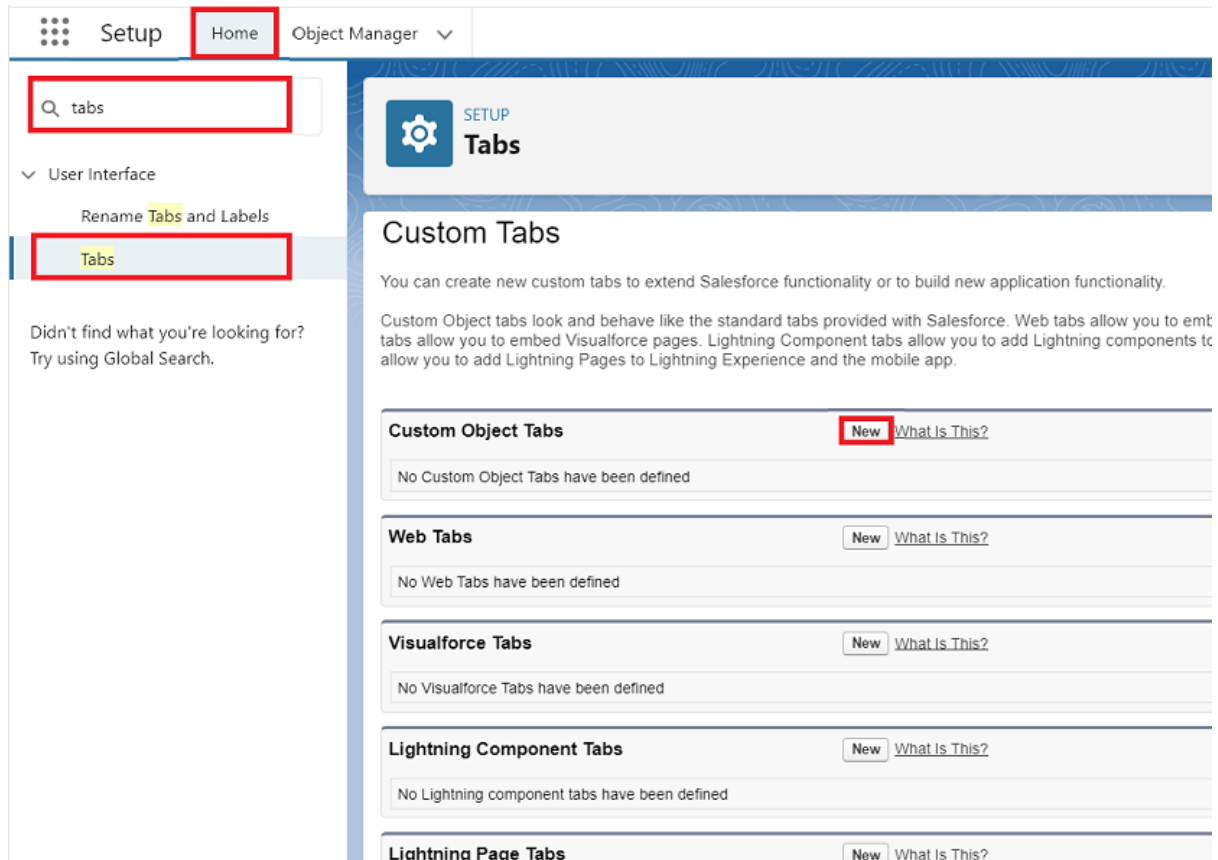
1. From the setup page → Click on Object Manager → Click on Create → Click on Custom Object.
 - 1) Enter the label name→ Vehicle Dealer
 - 2) Plural label name→ Vehicle Dealers
 - 3) Enter Record Name Label and Format
 - Record Name → Dealer Name
 - Data Type → Text
2. Click on Allow reports,
3. Allow search → **Save**

Activity 3: Creating remaining objects

Milestone 3 - Tabs

Activity 1: Creating a Custom Tab(Vehicle)

1. Go to setup page → type Tabs in Quick Find bar → click on tabs → New (under custom object tab)



2. Select Object(Vehicle) → Select any tab style → Next (Add to profiles page) keep it as default → Next (Add to Custom App) keep it as default → Save.

Setup Home Object Manager

Q: tab

Feature Settings

Analytics

Tableau

Tableau Embedding

Decision Tables

User Interface

Loaded Console Tab Limit

Rename Tabs and Labels

Tabs

Didn't find what you're looking for? Try using Global Search.

SETUP Tabs

New Custom Object Tab

Step 1. Enter the Details Step 1 of 3

Choose the custom object for this new custom tab. Fill in other details.

New Custom Object Tab

Select an existing custom object or create a new custom object now.

Object: --None--

Tab Style: --None--

(Optional) Choose a Home Page Custom Link to show as a splash page the first time your users click on this tab.

Splash Page Custom Link: --None--

Enter a short description.

Description:

Next Cancel

Activity 2:

Now create tabs for Other objects as well which we have created above Objects.

Milestone 4- The Lightning App:

Activity 1: Create a Lightning App

To create a lightning app page:

1. Go to setup page → search “app manager” in quick find → select “app manager” → click on New lightning App.

App Manager | Salesforce

sbcom-5e-dev-ed.develop.lightning.force.com/lightning/setup/NavigationMenus/home

Setup Home Object Manager

Q: app manager

Apps

Lightning Experience App Manager

New Lightning App New Connected App

Clone App (Beta)

Quickly create new Lightning apps by cloning existing apps. To use the beta feature, indicate that you've read all legal requirements and agree to participate by toggling Enable App Cloning. See additional details and terms in the [Winter '23 release notes](#).

Enable App Cloning: ☐

35 Items • Sorted by App Name • Filtered by All app menu items • Tab Set Type

App Name	Developer Name	Description	Last Modified	App Type	VL
1 All Tabs	AllTabs		04/12/2022, 10:13 am	Classic	
2 Analytics Studio	Insights	Build CRM Analytics dashboards and apps	04/12/2022, 10:13 am	Classic	
3 App Launcher	AppLauncher	App Launcher tabs	04/12/2022, 10:13 am	Classic	
4 Bolt Solutions	LightningBolt	Discover and manage business solutions designed for your industry.	04/12/2022, 10:13 am	Lightning	
5 Chatter Desktop	ChatterDesktop	Chatter Desktop is an Adobe AIR-based desktop application that lets Chatter users stay connecte...	23/12/2022, 4:04 pm	Connected (Managed)	
6 Chatter Mobile for BlackBerry	ChatterforBlackBerry	The Salesforce.com Chatter Mobile app lets you access Chatter data on the go. Use it to view fee...	23/12/2022, 4:05 pm	Connected (Managed)	
7 College Management System	CollegeMgmt	demo app	08/12/2022, 4:16 pm	Lightning	
8 Community	Community	Salesforce CRM Communities	04/12/2022, 10:13 am	Classic	
9 Content	Content	Salesforce CRM Content	04/12/2022, 10:13 am	Classic	
10 Data Manager	DataManager	Use Data Manager to view limits, monitor usage, and manage recipes.	04/12/2022, 10:13 am	Lightning	

2. Fill the app name in app details and branding as follow
App Name : WhatNext Vision Motors

Developer Name : this will auto populated

Description : Give a meaningful description

Image : optional (if you want to give any image you can otherwise not mandatory)

Primary color hex value : keep this default

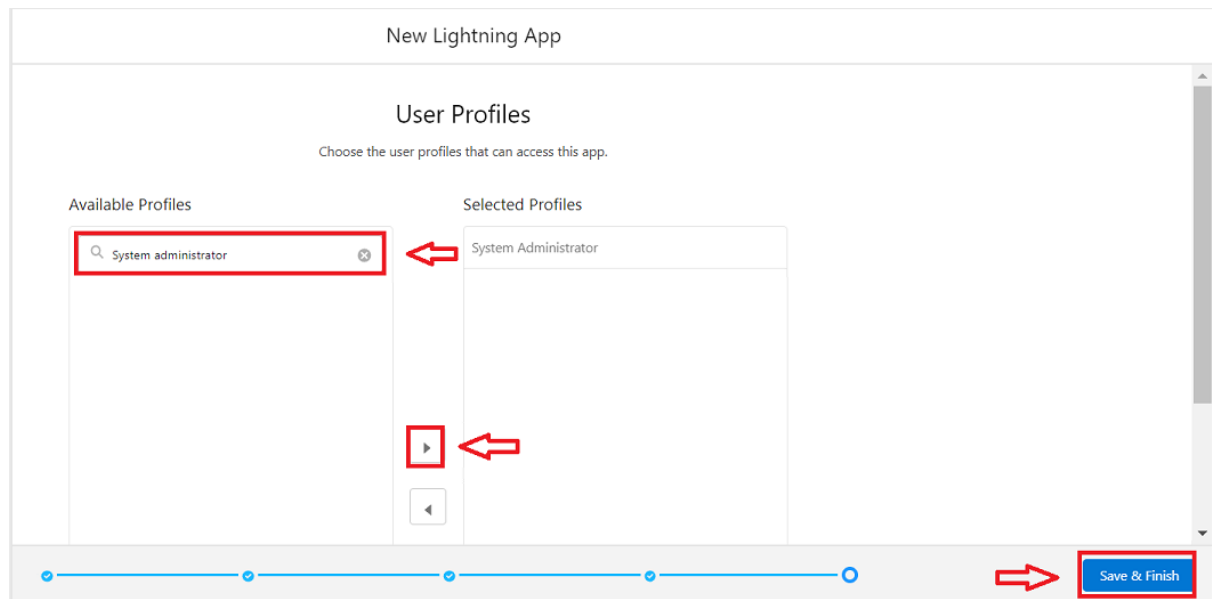
3. Then click Next → (App option page) keep it as default → Next → (Utility Items) keep it as default → Next.

4. To Add Navigation Items:

Search the items in the search bar(Vehicle, Dealer, Customer, Order, Test Drive, Service Request, Reports, Dashboard) from the search bar and move it using the arrow button → Next.

Note: select the custom object which we have created in the previous activity.

5. To Add User Profiles:



Search profiles (System administrator) in the search bar → click on the arrow button → save & finish.

Milestone 5 : Fields & Relationships

Key Fields for Each Object

1. Vehicle__c (Custom Object)

- **Vehicle Name**(Text)
- **Vehicle_Model__c** (Picklist: Sedan, SUV, EV, etc.)
- **Stock_Quantity__c** (Number)
- **Price__c** (Currency)
- **Dealer__c** (Lookup to Dealer__c)
- **Status__c** (Picklist: Available, Out of Stock, Discontinued)

2. Vehicle_Dealer__c (Custom Object)

- **Dealer_Name__c** (Text)
- **Dealer_Location__c** (Text)
- **Dealer_Code__c** (Auto Number)
- **Phone__c** (Phone)
- **Email__c** (Email)

3. Vehicle_Order__c (Custom Object)

- **Vehicle_Customer__c** (Lookup to Vehicle_Customer__c)
- **Vehicle__c** (Lookup to Vehicle__c)
- **Order_Date__c** (Date)
- **Status__c** (Picklist: Pending, Confirmed, Delivered, Canceled)

4. Vehicle_Customer__c (Custom Object)

- **Customer_Name__c** (Text)
- **Email__c** (Email)
- **Phone__c** (Phone)
- **Address__c** (Text)
- **Preferred_Vehicle_Type__c** (Picklist: Sedan, SUV, EV, etc.)

5. Vehicle_Test_Drive__c (Custom Object)

- **Vehicle_Customer__c** (Lookup to Vehicle_Customer__c)
- **Vehicle__c** (Lookup to Vehicle__c)

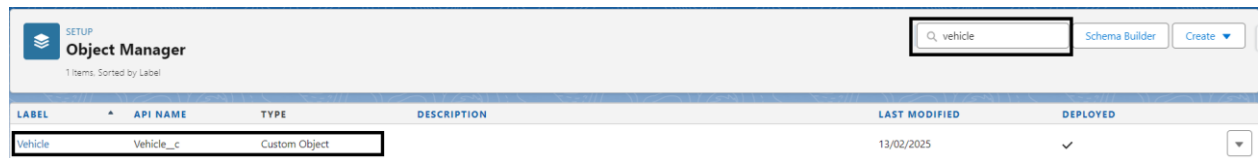
- **Test_Drive_Date__c** (Date)
- **Status__c** (Picklist: Scheduled, Completed, Canceled)

6. Vehicle_Service_Request__c (Custom Object)

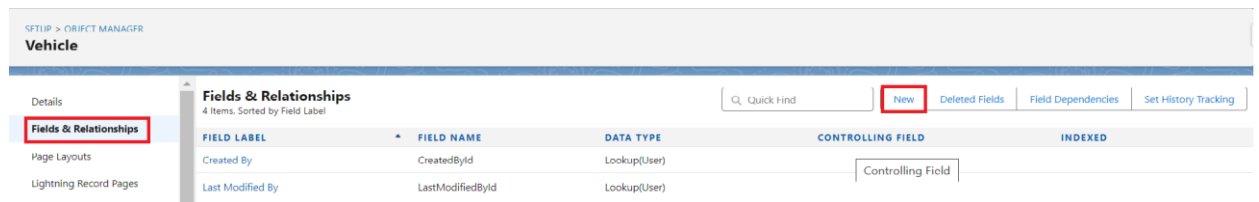
- **Vehicle_Customer__c** (Lookup to Vehicle_Customer__c)
- **Vehicle__c** (Lookup to Vehicle__c)
- **Service_Date__c** (Date)
- **Issue_Description__c** (Text)
- **Status__c** (Picklist: Requested, In Progress, Completed)

Activity 1: Creating a Field in Vehicle Object

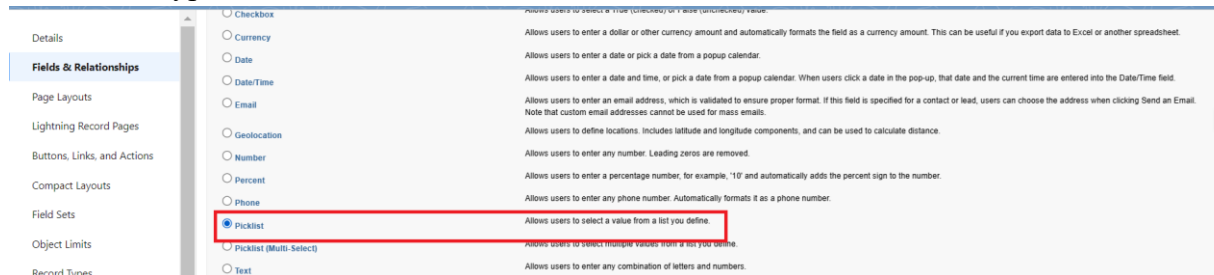
1. Go to setup → click on Object Manager → type object name(Vehicle) in quick find bar→ click on the object.



2. Now click on “Fields & Relationships” → New



3. Select Data type as “PickList”.



4. Click on Next

The screenshot shows the 'New Custom Field' configuration for the 'Vehicle' object. The 'Fields & Relationships' sidebar is on the left. The main area is titled 'Step 2. Enter the details'. The 'Field Label' is 'Vehicle Model'. Under 'Values', the option 'Enter values, with each value separated by a new line' is selected. A text box contains the values 'Sedan', 'SUV', and 'etc'. The 'Field Name' is 'Vehicle_Model'. The 'Next' button is highlighted with a red box.

5. Fill the above as following:
 - Field Label: Vehicle Modal
 - Fill PickList Values
 - Click on Next → Next → Save and new.

Activity 2 : Creating Stock Quantity field in vehicle object

1. Repeat the steps 1 & 2 from the activity 1
2. Select Data type as “Number”.

The screenshot shows the 'Data Type' selection screen. The 'Number' option is selected and highlighted with a red box. Other options include Checkbox, Currency, Date, Date/Time, Email, Geolocation, Percent, and Phone.

3. Click on Next

The screenshot shows the 'Stock Quantity' field configuration. The 'Field Label' is 'Stock Quantity'. The 'Field Name' is 'Stock_Quantity'. The 'Length' is set to 18 and 'Decimal Places' is set to 0. The 'Next' button is highlighted with a red box.

4. Fill the above as following:
 - Field Label: Stock Quality
 - Click on Next → Next → Save and new.

Activity 3 : Creating a Lookup Relationship in Vehicle Object to Dealer Object.

1. Select DataType as Lookup Relationship.

SETUP > OBJECT MANAGER
Vehicle

Details
Fields & Relationships
Page Layouts
Lightning Record Pages
Buttons, Links, and Actions
Compact Layouts
Field Sets
Object Limits
Record Types
Related Lookup Filters
Restriction Rules
Scoping Rules

New Custom Field

Step 1. Choose the field type

Specify the type of information that the custom field will contain.

Data Type

☐ None Selected Select one of the data types below.

☐ Auto Number A system-generated sequence number that uses a display format you define. The number is automatically incremented for each new record.

☐ Formula A read-only field that derives its value from a formula expression you define. The formula field is updated when any of the source fields change.

☐ Roll-Up Summary A read-only field that displays the sum, minimum, or maximum value of a field in a related list or the record count of all records listed in a related list.

☒ **Lookup Relationship** Creates a relationship that links this object to another object. The relationship field allows users to click on a lookup icon to select a value from a popup list. The other object is the source of the values in the list.

☐ Master-Detail Relationship Creates a special type of parent-child relationship between this object (the child, or "detail") and another object (the parent, or "master") where:

- The relationship field is required on all detail records.
- The ownership and sharing of a detail record are determined by the master record.
- When a user deletes the master record, all detail records are deleted.

Next

2. Click Next

3. Select Dealer Object Form Related Filed.

Details
Fields & Relationships
Page Layouts
Lightning Record Pages
Buttons, Links, and Actions
Compact Layouts
Field Sets
Object Limits
Record Types
Related Lookup Filters
Restriction Rules

New Relationship

Step 2. Choose the related object

Select the other object to which this object is related.

Related To: Dealer

Previous Next Cancel

New Relationship

Step 3. Enter the label and name for the lookup field

Field Label: Dealer

Field Name: Dealer

Description:

Help Text:

Previous Next Cancel

New Relationship

Step 4. Establish field-level security for reference field

Field Label	Dealer
Data Type	Lookup
Field Name	Dealer
Description	
Select the profiles to which you want to grant edit access to this field via field-level security. The field will be hidden from all profiles if you do not add it to field-level security.	
Field-Level Security for Profile	☒ Visible ☐ Read-Only
Analytics Cloud Integration User	☒ ☐
Analytics Cloud Security User	☒ ☐
Contract Manager	☒ ☐
Cross Org Data Proxy User	☒ ☐
Custom Marketing Profile	☒ ☐

Previous Next Cancel

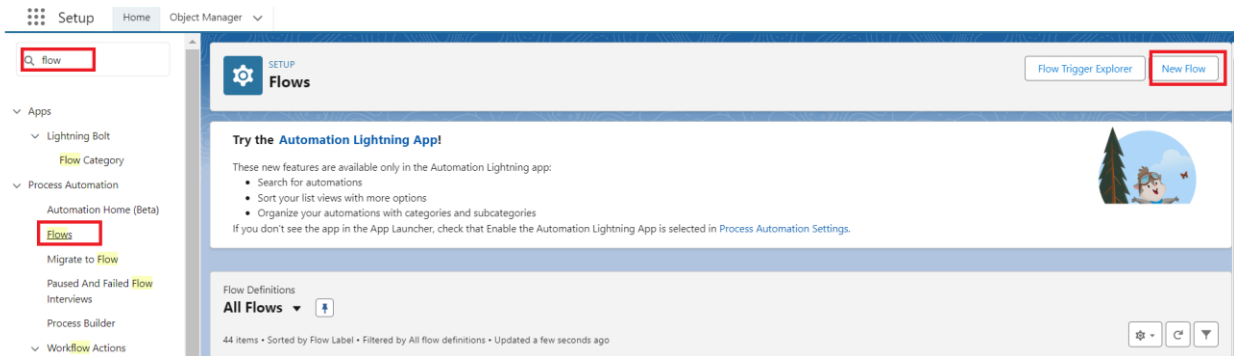
4. Click Next and Save.

Activity 4 : Creating Remaining Objects and Fields.

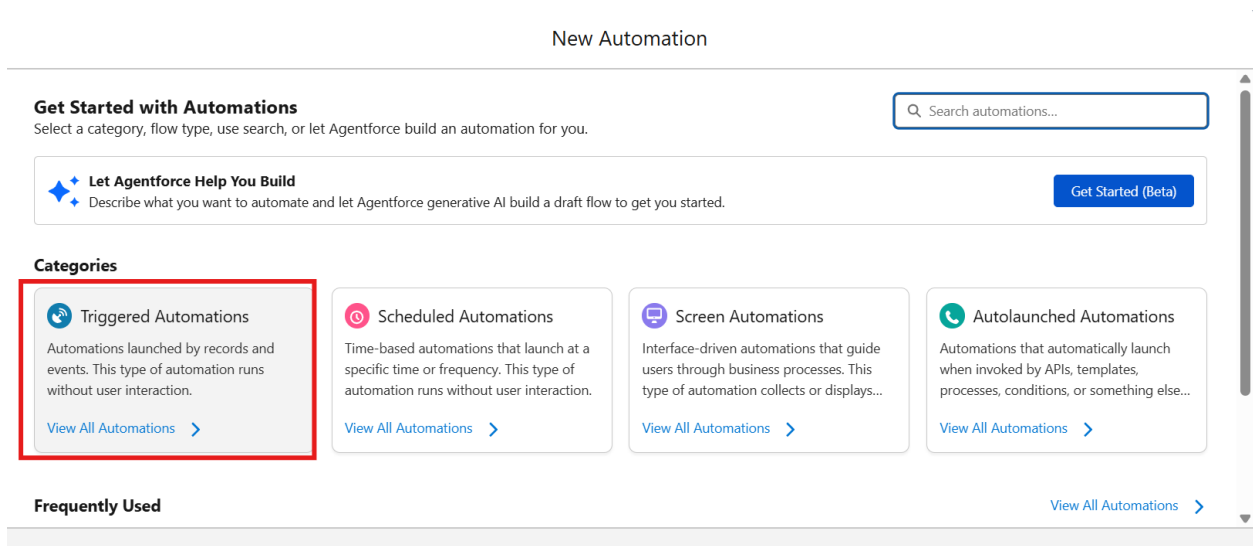
MileStone 6 :

Flow 1 : Creating a record triggered flow to assign nearest dealer to the customer's location

Step 1 : In Quick Find, type Flows and click on Flows. Click New Flow.



Step 2 : Select Triggered Automation From Flow.



Step 3 : Select Record-Triggered Flow

New Automation

**Triggered Automations**
Showing 12 of 97 results

Filter By ☒ Triggered ☐ Scheduled ☐ Screen ☐ Autolaunched

Types (6)

**Record-Triggered Flow**
Launches when a record is created, updated, or deleted. This autolaunched flow runs in the background.

**Platform Event—Triggered Flow**
Launches when a platform event message is received. This autolaunched flow runs in the background.

**Automation Event-Triggered Flow**
Launches when an automation event is received. This type of flow runs in the background without user interaction.

**Activation-Triggered Flow**
Launches when an activation is published. This type of flow runs in the background without user interaction.

**Record-Triggered Orchestration**
Launches when a record is created or updated. An orchestration lets you create a multi-step, multi-user

**Record-Triggered Flow Approval Process**
Launches when a record is created or updated. A flow approval process lets you create a multi-step, multi-user

[Back](#)

Step 4 : Select Vehicle Order Object

Trigger the Flow When : Select Record is Created.

Set Entry Condition :

All Conditions Are Met (AND)

Filed : Status__c

Operator : Equals

Value : Pending

**Start**
Record-Triggered Flow

Object: **Vehicle Order** [Edit](#)

Trigger: **A record is created**

Conditions: **1**

Optimize for: **Actions and Related Records**

[+ Add Scheduled Paths \(Optional\)](#)

[Open Flow Trigger Explorer for Vehicle Order](#)

Run Immediately

**Get Customer Information.**
Get Records

**Get Nearest Dealer**
Get Records

**Configure Start**

Configure Trigger

***Trigger the Flow When:**

☒ A record is created

☐ A record is updated

☐ A record is created or updated

☐ A record is deleted

Set Entry Conditions

Specify entry conditions to reduce the number of records that trigger the flow and the number of times the flow is executed. Minimizing unnecessary flow executions helps to conserve your org's resources.

If you create a flow that's triggered when a record is updated, we recommend first defining entry conditions. Then select the **Only when a record is updated to meet the condition requirements** option for When to Run the Flow for Updated Records.

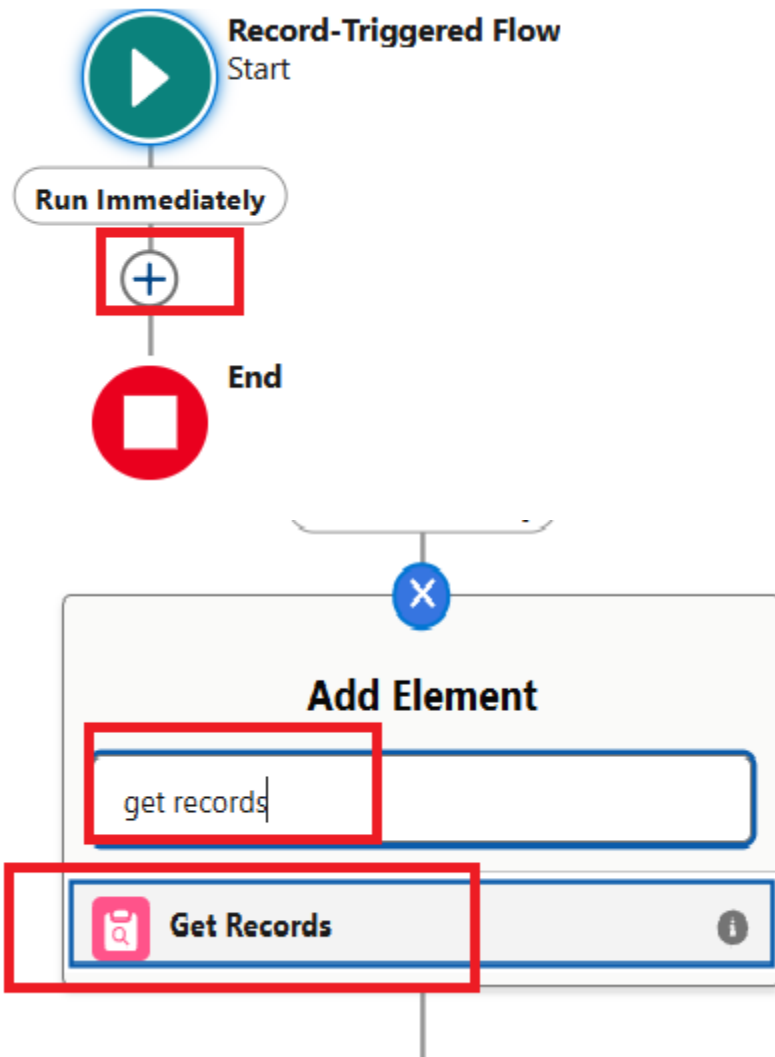
Condition Requirements

All Conditions Are Met (AND)

Field	Operator	Value
Vehicle_Customer__c	Is Null	☐ True ☒ False ×

[+ Add Condition](#)

Step 5 : Click + → Select **Get Records**



Step 6 : **Label** : Get Customer Information

Object : Vehicle Customer

Condition :

Field : Id

Operator : Equals

Value : {!\$Record.Vehicle_Customer__c}

 Get Records

* Label

Get Customer Information.

* API Name

Get_Customer_Information

Description

Get Records of This Object

* Object

Vehicle Customer

Filter Vehicle Customer Records

Condition Requirements

All Conditions Are Met (AND)

Field

Id

Operator

Equals

Value

Aa \$Record > Vehicle Customer X

+ Add Condition

Sort Vehicle Customer Records

Step 7 : Click + → Select **Get Records**

Label : Get Nearest Dealer

Object : Vehicle Dealer

Condition :

Field : Dealer_Location__c

Operator : Equals

Value : {!Get_Customer_Information.Address__c}



Get Records

*Label

Get Nearest Dealer

*API Name 

Get_Nearest_Dealer

Description

Get Records of This Object

*Object

Vehicle Dealer

Filter Vehicle Dealer Records

Condition Requirements

All Conditions Are Met (AND) 

Field

Dealer_Location__c

Operator

Equals 

Value

 Vehicle Customer from Get_Cu... 

[+ Add Condition](#)

Step 8 : Click + → Select **Update Records**

Label : Assign Dealer to Order

*How to Find Records to Update and Set Their Values : Use the IDs and all field values from a record or record collection

Select Record(s) to Update : {!Get_Nearest_Dealer}

 Update Records

* Label

Assign Dealer to Order

* API Name ⓘ

Assign_Dealer_to_Order

Description

* How to Find Records to Update and Set Their Values

☐ Use the vehicle order record that triggered the flow

☐ Update records related to the vehicle order record that triggered the flow

☒ Use the IDs and all field values from a record or record collection

☐ Specify conditions to identify records, and set fields individually

Select Record(s) to Update

Record or Record Collection

 Vehicle Dealer from Get Nearest Dealer X

Make sure that each record has an ID. Otherwise the flow can't find the records to update, and it fails. ⓘ

Step 9 : Click Save and Give label Name and Activate Flow.

Label Name : Auto Assign Dealer

Save as

Save As

A New Version

* Flow Label

Auto Assign Dealer

* Flow API Name ⓘ

Auto_Assign_Dealer

Description

Show Advanced

Cancel

Save

Step 10 : Activate Flow

Last saved on 13/2/2025, 02:52 am

Deactivated

Run

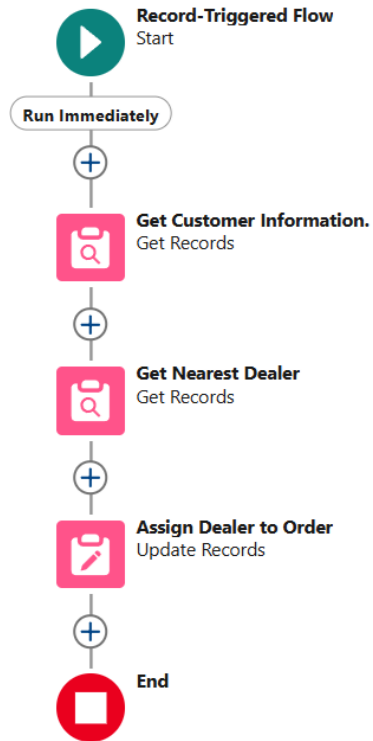
Debug

View Tests

Save As New Version

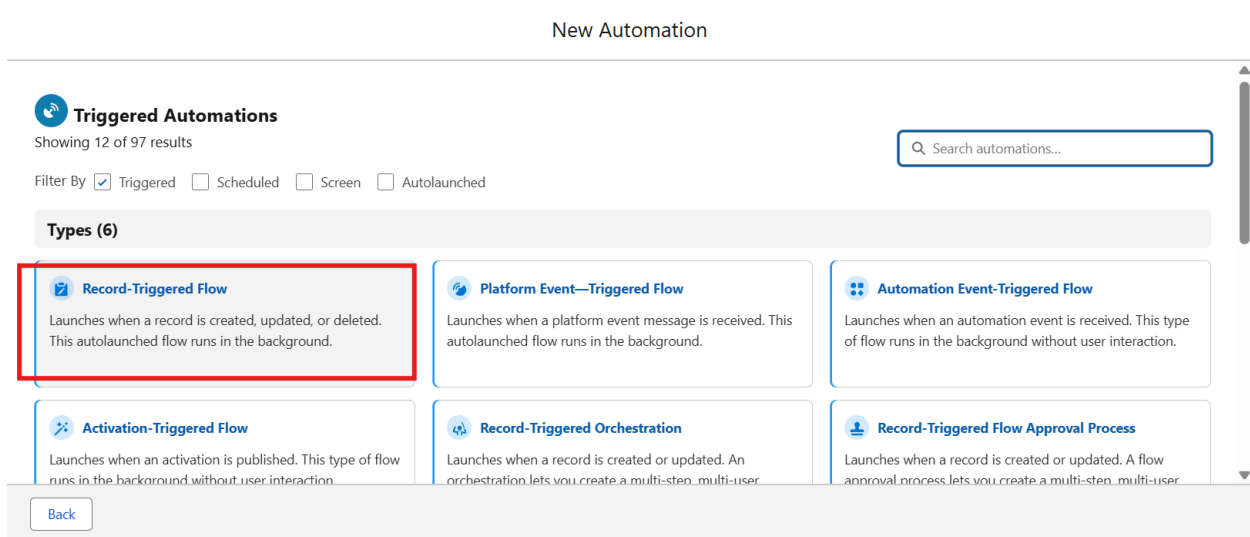
Save

Activate



Flow 2 : Creating record triggered flow to send an email to the customer reminding about the test drive.

Step 1 : Select Record-Triggered Flow



Step 2 : Select Vehicle Test Drive Object

Trigger the Flow When : A record is created or updated

Set Entry Condition :

All Conditions Are Met (AND)

Filed : Status__c

Operator : Equals

Value : Scheduled

Step 3 : Click + Add Scheduled Paths (below the trigger).

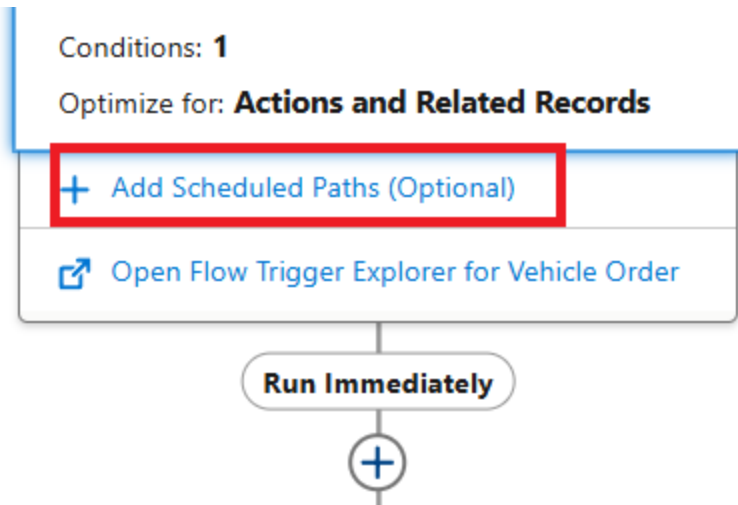
Label: Reminder Before Test Drive.

Time Source: Test_Drive_Date__c

Offset Number: 1.

Offset Options: Days Before.

Click Done.



Configure Scheduled Paths

SCHEDULED PATHS



Run Immediately

Add a scheduled path if you want a flow path to run a certain amount of time after the triggering record is created or updated. That time can be based on the triggering event or on a specified date or date/time field in the record. A separate flow interview runs for each scheduled path. Each scheduled path interview is queued to run at the scheduled time, but can be delayed until system resources become available. Path labels appear on connectors in the canvas.

Reminder Before Test Drive.

SCHEDULED PATH DETAILS

[Delete Path](#)

* Path Label

Reminder Before Test Drive.

* API Name ⓘ

Reminder_Before_Test_Drive

* Time Source

Vehicle_Test_Drive__c: Test Drive Date

* Offset Number

1

* Offset Options ⓘ

Days Before

> Show advanced options

Step 4 :

1. Click + Add Element → Get Records.
2. Label: Get Customer Information.
3. Object: Vehicle_Customer__c.
4. Filter Conditions: Id = {!\$Record.Customer__c}.
5. How Many Records to Store: Select Only the first record.
6. How to Store Record Data: Choose Automatically store all fields.

Get Records

* Label

Get Customer Information

* API Name ⓘ

Get_Customer_Information

Description

Get Records of This Object

Object

Vehicle Customer

Filter Vehicle Customer Records

Condition Requirements

All Conditions Are Met (AND) ▼

Field

Operator

Value

Id

Equals ▼

Aa \$Record > Vehicle Customer X

+ Add Condition

Sort Vehicle Customer Records

1. Click + Add Element → Action.
2. Action Type: Send Email
3. Label: Send Test Drive Reminder.
4. Subject: "Reminder: Your Test Drive is Tomorrow!".
5. Recipient Address List : {!Get_Customer_Information.Email__c}
6. Rich-Text-Formatted Body : True

 Send Email

* Label

Send Test Drive Reminder.

* API Name ⓘ

Send_Test_Drive_Reminder

Description

 **Send Email** ⓘ

emailSimple-emailSimple

7. Body : Create Variable

New Resource

* Resource Type

Select...

- Variable
Store a value that can be used and changed throughout the flow.
- Constant
Store a value that can be used but not changed throughout the flow.
- Formula
Calculate a value when the formula is used in the flow.
- Text Template**
Store text that can be used and changed throughout the flow.
- Stage
Identify different phases in the flow to track user progress.

8 . Api Name : EmailSent

Edit Text Template

* API Name ⓘ

EmailSent

Description

* Body ⓘ

Insert a resource... 🔍 View as Rich Text ▼

Dear {!Get_Customer_Information.Name},

This is a reminder that your test drive for {!\$Record.Vehicle__c} is scheduled for tomorrow. If you need to reschedule, contact us at [Support Email]. Thank you, WhatNext Vision Motors Team"

Salesforce Sans ▼ 12 ▼  **B** *I* U          

Cancel Done

Send Email

A_a Body 

 EmailSent 

 Included

A_a CC Recipient Address List

 Not Included

A_a Email Template ID

 Not Included

 Log Email on Send

 Not Included

A_a Recipient Address Collection

 Not Included

Step 6 : Click save

Label Name : Test Drive Reminder

Save as

Save As

A New Version 

Flow Label

Test Drive Reminder

* Flow API Name 

Test_Drive_Reminder

Description

[Show Advanced](#)

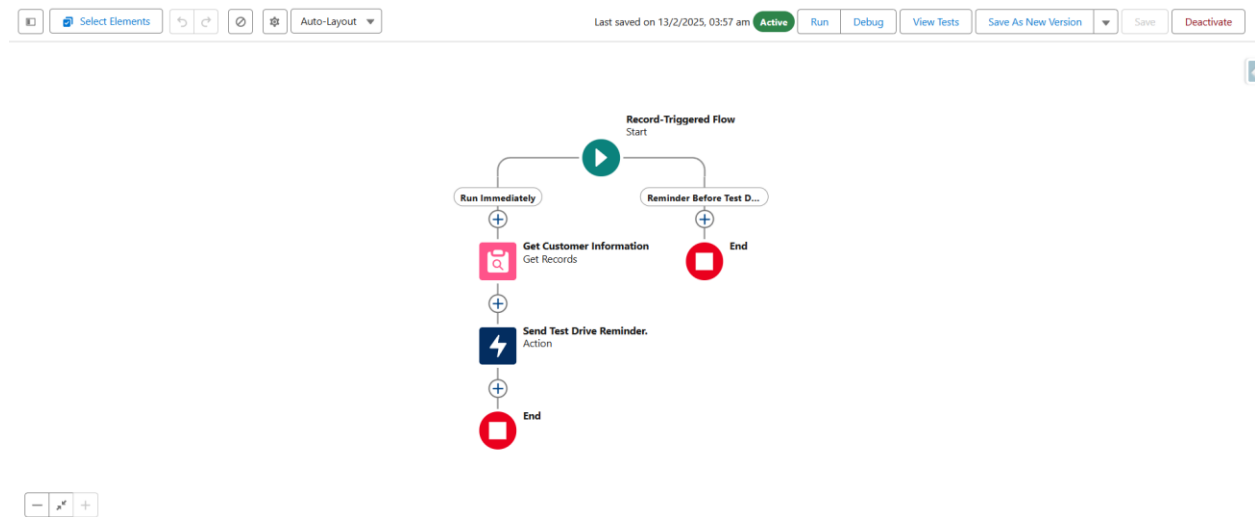
Cancel

Save

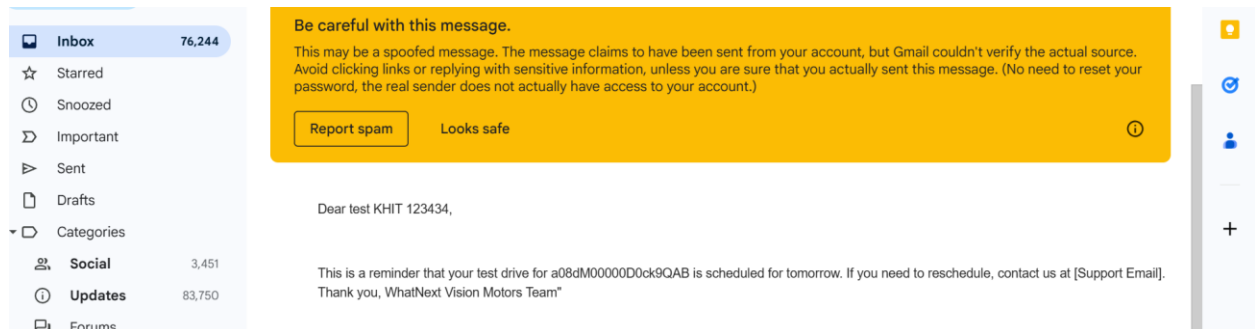
Step 7 : Activate Flow



Flow Page After Creation :



Test your Flow :





Send Email

A_a Recipient Address List

A_a ...er from Get Customer Information > Email



Included

A_a Recipient ID

Vehicle Customer from Get Customer Information > Email



Not Included

A_a Related Record ID



Not Included

Rich-Text-Formatted Body

True



Included

A_a Sender Email Address



Not Included

A_a Sender Type



Not Included

A_a Subject

Reminder: Your Test Drive is Tomorrow!



Included

Use Line Breaks

True

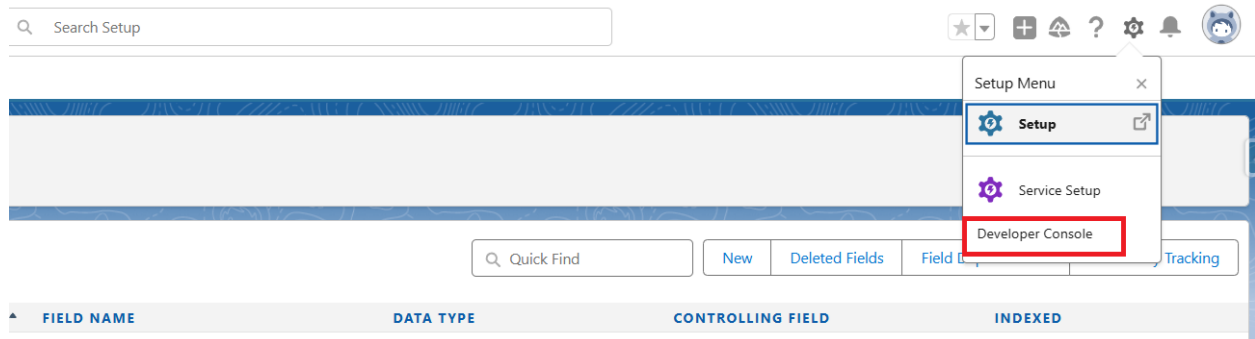


Included

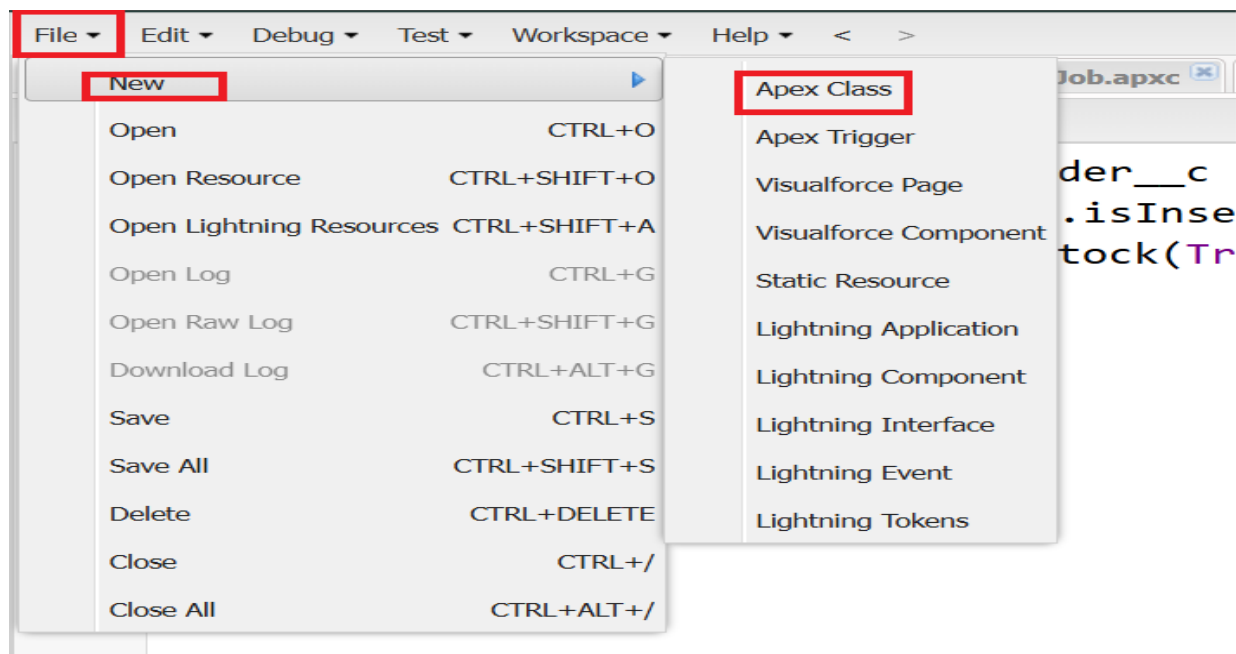
MileStone 7 :

Create Apex and Trigger Batch Jobs :

Step 1 : Click Developer Console From Gear icon

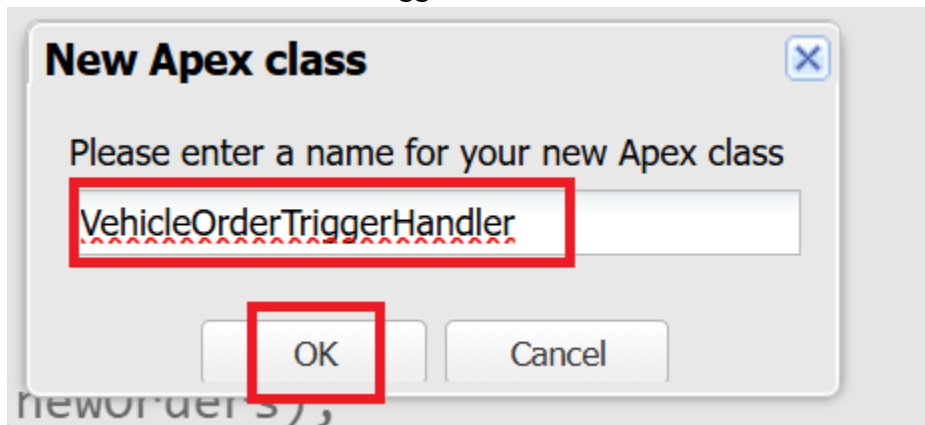


Step 2 : click File then Select New then Select Apex.



Step 3 : Give Name For Apex Class

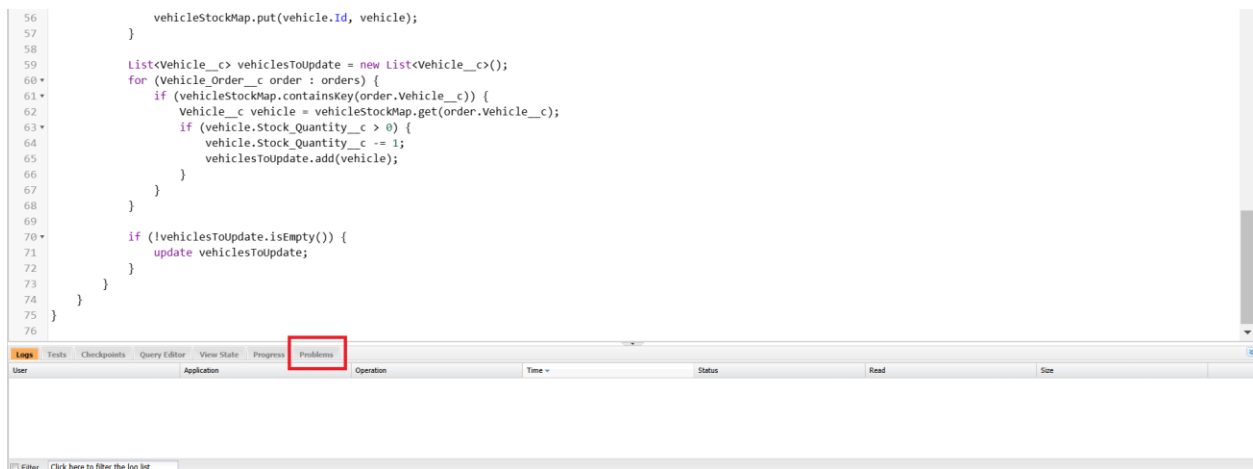
Class Name : VehicleOrderTriggerhandler



Step 4 : Write Apex Code

```
File • Edit • Debug • Test • Workspace • Help • <
OrderTriggerHandler.apex • OrderTrigger.apex • InventoryBatchJob.apex • VehicleOrderTriggerHandler.apex • VehicleOrderTrigger.apex
Code Coverage: None • API Version: 62
Go To

1 • public class VehicleOrderTriggerHandler {
2
3 •     public static void handleTrigger(List<Vehicle_Order__c> newOrders, Map<Id, Vehicle_Order__c> oldOrders, Boolean isBefore, Boolean isAfter, Boolean isInsert, Boolean isUpdate) {
4 •         if (isBefore) {
5 •             if (isInsert || isUpdate) {
6 •                 preventOrderIfOutOfStock(newOrders);
7 •             }
8 •         }
9
10 •         if (isAfter) {
11 •             if (isInsert || isUpdate) {
12 •                 updateStockOnOrderPlacement(newOrders);
13 •             }
14 •         }
15 •     }
16
17 •     // Method to prevent orders when the vehicle is out of stock
18 •     private static void preventOrderIfOutOfStock(List<Vehicle_Order__c> orders) {
19 •         Set<Id> vehicleIds = new Set<Id>();
20 •         for (Vehicle_Order__c order : orders) {
21 •             if (order.Vehicle__c != null) {
22 •                 vehicleIds.add(order.Vehicle__c);
23 •             }
24 •         }
25
26 •         if (!vehicleIds.isEmpty()) {
27 •             Map<Id, Vehicle__c> vehicleStockMap = new Map<Id, Vehicle__c>();
28 •             for (Vehicle__c vehicle : [SELECT Id, Stock_Quantity__c FROM Vehicle__c WHERE Id IN :vehicleIds]) {
29 •                 vehicleStockMap.put(vehicle.Id, vehicle);
30 •             }
31
32 •             for (Vehicle_Order__c order : orders) {
33 •                 if (vehicleStockMap.containsKey(order.Vehicle__c)) {
34 •                     Vehicle__c vehicle = vehicleStockMap.get(order.Vehicle__c);
35 •                     if (vehicle.Stock_Quantity__c <= 0) {
36 •                         order.addError('This vehicle is out of stock. Order cannot be placed.');
```



Source Code :

```
public class VehicleOrderTriggerHandler {
```

```
    public static void handleTrigger(List<Vehicle_Order__c> newOrders, Map<Id,
Vehicle_Order__c> oldOrders, Boolean isBefore, Boolean isAfter, Boolean isInsert, Boolean
isUpdate) {
```

```
        if (isBefore) {
            if (isInsert || isUpdate) {
                preventOrderIfOutOfStock(newOrders);
            }
        }
    }
```

```
        if (isAfter) {
            if (isInsert || isUpdate) {
                updateStockOnOrderPlacement(newOrders);
            }
        }
    }
}
```

// Method to prevent orders when the vehicle is out of stock

```
private static void preventOrderIfOutOfStock(List<Vehicle_Order__c> orders) {
    Set<Id> vehicleIds = new Set<Id>();
    for (Vehicle_Order__c order : orders) {
        if (order.Vehicle__c != null) {
            vehicleIds.add(order.Vehicle__c);
        }
    }
}
```

```

    if (!vehicleIds.isEmpty()) {
        Map<Id, Vehicle__c> vehicleStockMap = new Map<Id, Vehicle__c>();
        for (Vehicle__c vehicle : [SELECT Id, Stock_Quantity__c FROM Vehicle__c WHERE
Id IN :vehicleIds]) {
            vehicleStockMap.put(vehicle.Id, vehicle);
        }

        for (Vehicle_Order__c order : orders) {
            if (vehicleStockMap.containsKey(order.Vehicle__c)) {
                Vehicle__c vehicle = vehicleStockMap.get(order.Vehicle__c);
                if (vehicle.Stock_Quantity__c <= 0) {
                    order.addError('This vehicle is out of stock. Order cannot be placed.');
```

```

                }
            }
        }
    }
}

// Method to update vehicle stock when an order is placed
private static void updateStockOnOrderPlacement(List<Vehicle_Order__c> orders) {
    Set<Id> vehicleIds = new Set<Id>();
```

```

    for (Vehicle_Order__c order : orders) {
        if (order.Vehicle__c != null && order.Status__c == 'Confirmed') {
            vehicleIds.add(order.Vehicle__c);
        }
    }
}
```

```

    if (!vehicleIds.isEmpty()) {
        Map<Id, Vehicle__c> vehicleStockMap = new Map<Id, Vehicle__c>();
        for (Vehicle__c vehicle : [SELECT Id, Stock_Quantity__c FROM Vehicle__c WHERE
Id IN :vehicleIds]) {
            vehicleStockMap.put(vehicle.Id, vehicle);
        }
    }
}
```

```

    List<Vehicle__c> vehiclesToUpdate = new List<Vehicle__c>();
    for (Vehicle_Order__c order : orders) {
        if (vehicleStockMap.containsKey(order.Vehicle__c)) {
            Vehicle__c vehicle = vehicleStockMap.get(order.Vehicle__c);
```

```

        if (vehicle.Stock_Quantity__c > 0) {
            vehicle.Stock_Quantity__c -= 1;
            vehiclesToUpdate.add(vehicle);
        }
    }
}

if (!vehiclesToUpdate.isEmpty()) {
    update vehiclesToUpdate;
}
}
}
}

```

Vehicle Order
VC-0003

Vehicle Order Name

VC-0003

Vehicle Customer

test KHIT 123434

Vehicle

1erwe

Order Date

21/02/2025

Status

Pending

Owner

Abdul Trainer

Created By

Abdul Trainer, 13/02/2025, 5:34 am

By

Abdul Trainer, 13/02/2025, 5:41 am

We hit a snag.

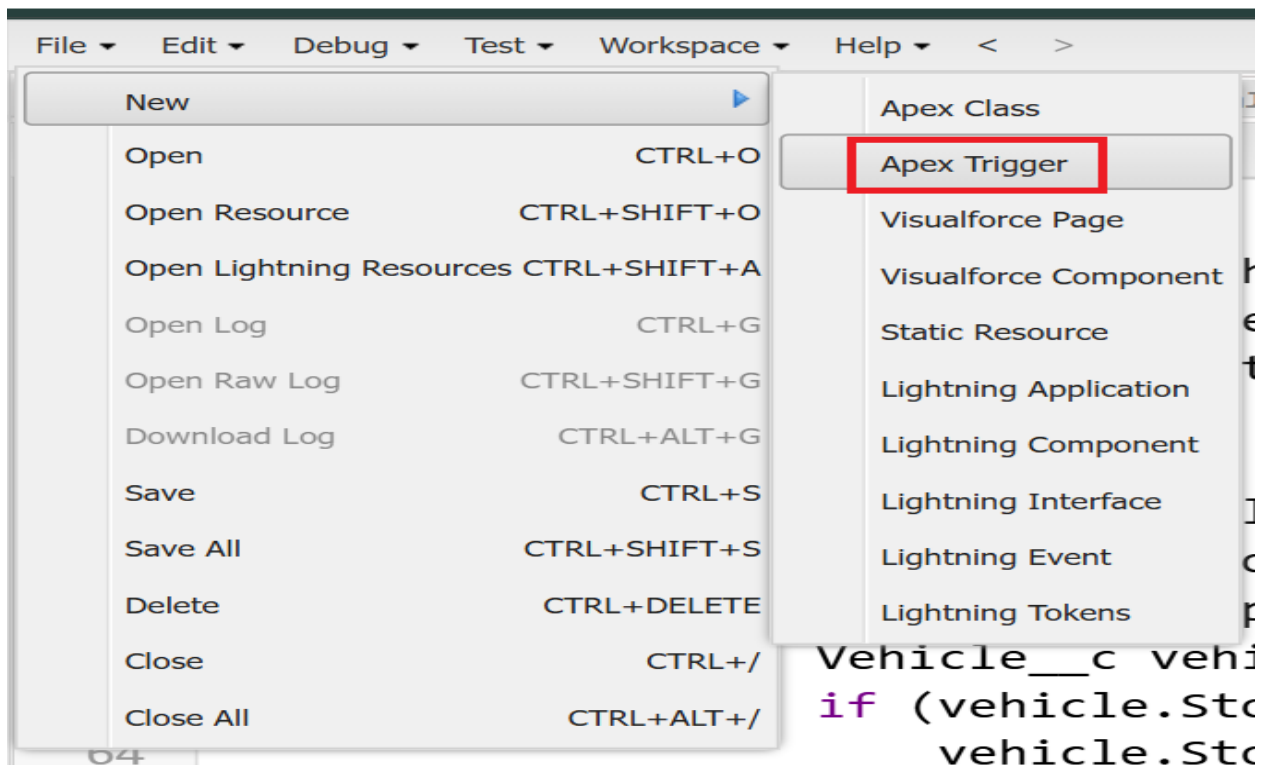
Review the errors on this page.

- This vehicle is out of stock. Order cannot be placed.

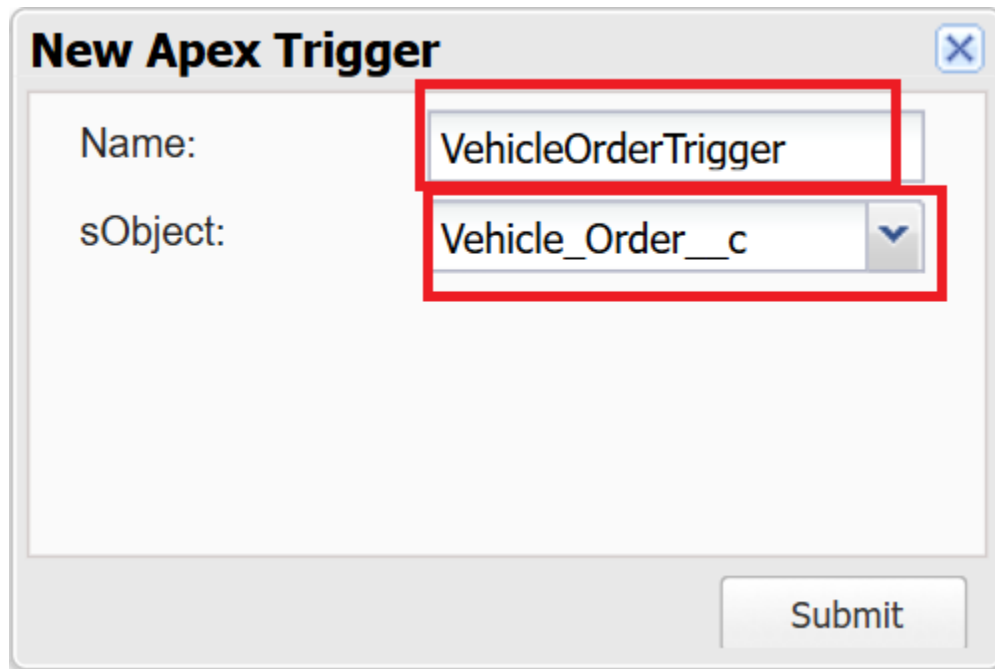
Cancel

Save

Step 5 : Write Trigger Handler



Step 6 : Writer Trigger Class Name and Select Vehicle Order Object



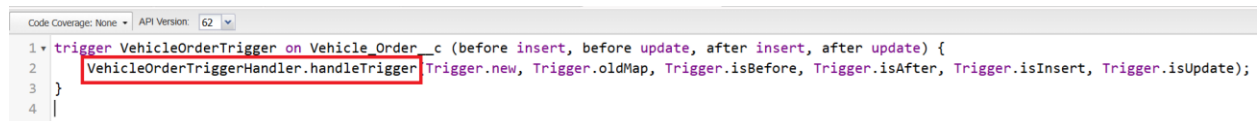
New Apex Trigger

Name: VehicleOrderTrigger

sObject: Vehicle_Order__c

Submit

Step 7 : Call Apex Class in Trigger Class



```
1 trigger VehicleOrderTrigger on Vehicle_Order__c (before insert, before update, after insert, after update) {
2     VehicleOrderTriggerHandler.handleTrigger Trigger.new, Trigger.oldMap, Trigger.isBefore, Trigger.isAfter, Trigger.isInsert, Trigger.isUpdate);
3 }
4
```

Step 8 : Create Batch Job

A customer places an order, but the vehicle is out of stock.

- The order remains pending.
- After new stock is added, the batch job updates the order to confirmed.

New Apex class

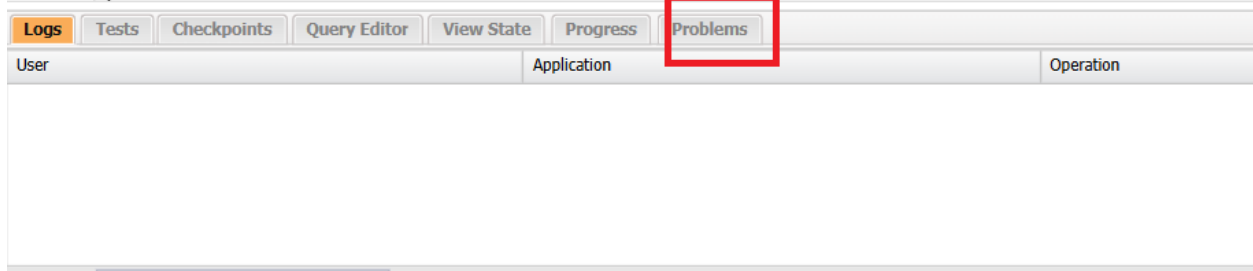
Please enter a name for your new Apex class

OK

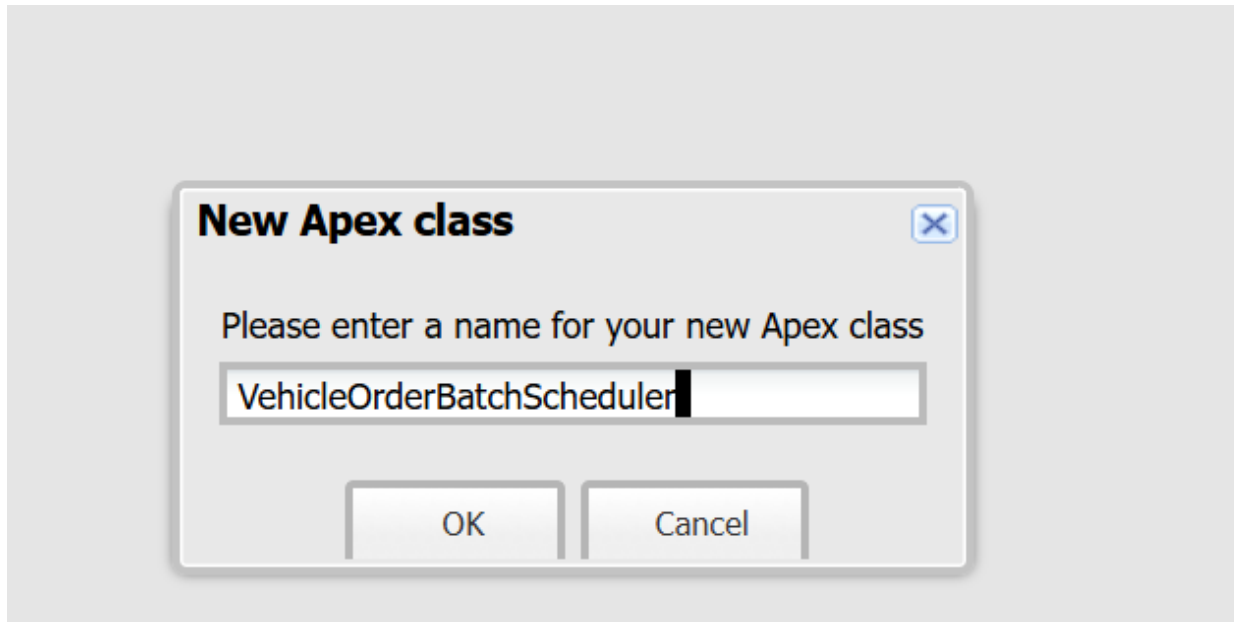
Cancel

```
Code Coverage: None | API Version: 62
1  global class VehicleOrderBatch implements Database.Batchable<sObject> {
2
3
4      global Database.QueryLocator start(Database.BatchableContext bc) {
5          return Database.getQueryLocator([
6              SELECT Id, Status__c, Vehicle__c
7              FROM Vehicle_Order__c
8              WHERE Status__c = 'Pending'
9          ]);
10     }
11
12     global void execute(Database.BatchableContext bc, List<Vehicle_Order__c> orderList) {
13         Set<Id> vehicleIds = new Set<Id>();
14
15         for (Vehicle_Order__c order : orderList) {
16             if (order.Vehicle__c != null) {
17                 vehicleIds.add(order.Vehicle__c);
18             }
19         }
20
21         if (!vehicleIds.isEmpty()) {
22             Map<Id, Vehicle__c> vehicleStockMap = new Map<Id, Vehicle__c>();
23             for (Vehicle__c vehicle : [SELECT Id, Stock_Quantity__c FROM Vehicle__c WHERE Id IN :vehicleIds]) {
24                 vehicleStockMap.put(vehicle.Id, vehicle);
25             }
26
27             List<Vehicle_Order__c> ordersToUpdate = new List<Vehicle_Order__c>();
28             List<Vehicle__c> vehiclesToUpdate = new List<Vehicle__c>();
29
30             for (Vehicle_Order__c order : orderList) {
31                 if (vehicleStockMap.containsKey(order.Vehicle__c)) {
```

```
30 ▾      if (vehicleStockMap.containsKey(order.Vehicle__c)) {
31          Vehicle__c vehicle = vehicleStockMap.get(order.Vehicle__c);
32 ▾      if (vehicle.Stock_Quantity__c > 0) {
33          order.Status__c = 'Confirmed';
34          vehicle.Stock_Quantity__c -= 1;
35          ordersToUpdate.add(order);
36          vehiclesToUpdate.add(vehicle);
37      }
38  }
39  }
40
41 ▾      if (!ordersToUpdate.isEmpty()) {
42          update ordersToUpdate;
43      }
44
45 ▾      if (!vehiclesToUpdate.isEmpty()) {
46          update vehiclesToUpdate;
47      }
48  }
49  }
50
51 ▾  global void finish(Database.BatchableContext bc) {
52      System.debug('Vehicle order batch job completed.');
```



Step 9 : Create Schedule Class then revoked Batch Class in Schedule Class

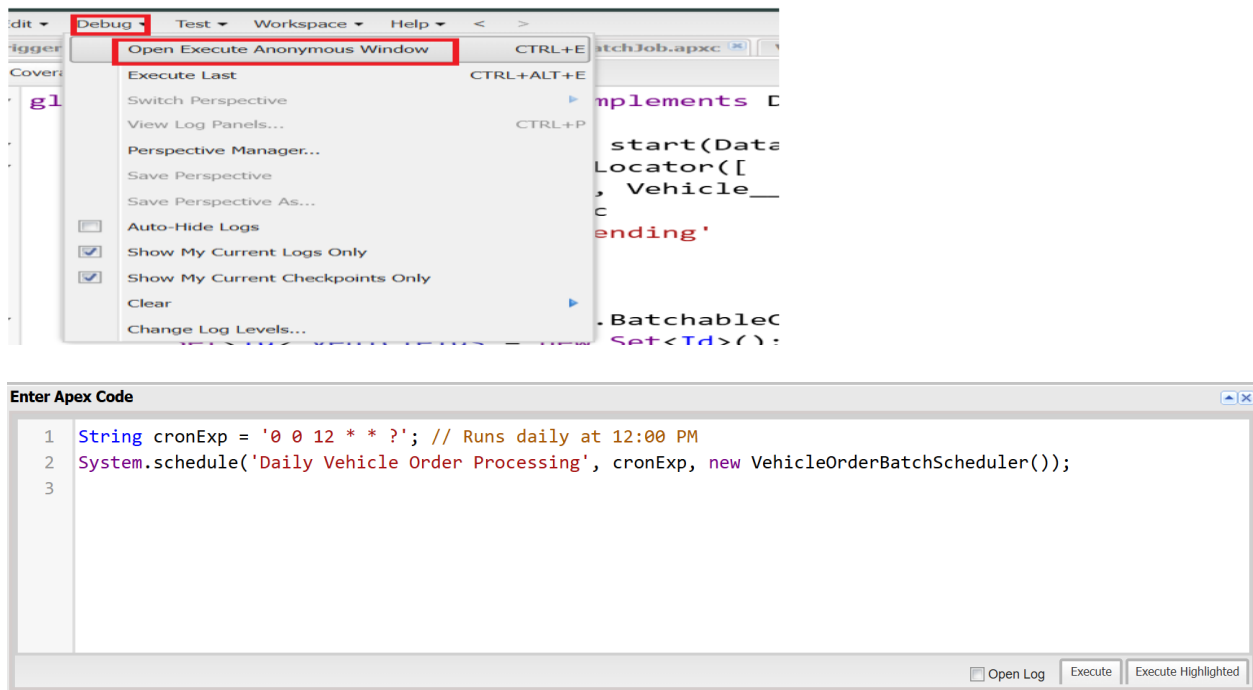


```
OrderTriggerHandler.apxc | OrderTrigger.apxt | InventoryBatchJob.apxc | VehicleOrderTriggerHandler.apxc | VehicleOrderTrigger.apxt | VehicleOrderBatchJob.apxc
Code Coverage: None | API Version: 62
1 global class VehicleOrderBatchScheduler implements Schedulable {
2     global void execute(SchedulableContext sc) {
3         VehicleOrderBatch batchJob = new VehicleOrderBatch();
4         Database.executeBatch(batchJob, 50);
5     }
6 }
7
```

Source Code :

```
global class VehicleOrderBatchScheduler implements Schedulable {
    global void execute(SchedulableContext sc) {
        VehicleOrderBatch batchJob = new VehicleOrderBatch();
        Database.executeBatch(batchJob, 50); // 50 is the batch size
    }
}
```

Step 10 : Schedule the Batch Job To run the batch job every night at midnight :



Source Code :

```
String cronExp = '0 0 12 * * ?'; // Runs daily at 12:00 PM
```

```
System.schedule('Daily Vehicle Order Processing', cronExp, new  
VehicleOrderBatchScheduler());
```

Step 11 : You Can Check where this Schedule Job is Running

Setup Home Object Manager

jobs

Environments

Jobs

Apex Flex Queue

Apex Jobs

Background Jobs

Bulk Data Load Jobs

Scheduled Jobs

Scheduled Jobs

Didn't find what you're looking for? Try using Global Search.

Scheduled Jobs

All Scheduled Jobs

The All Scheduled Jobs page lists all of the jobs scheduled by your users. Multiple job types may display on this page. You can delete scheduled jobs if you have the permission to do so.

Percentage of Scheduled Jobs Used: 2%
You have currently used 2 scheduled Apex jobs out of an allowed organization limit of 100 active or scheduled jobs. To learn about how this limit is calculated and what contributes to it see the [Lightning Platform Apex Limits](#) topic.

View: All Scheduled Jobs Create New View

A B C D E F G H I J K L M N O P Q R S T U V W X Y Z Other All

Action	Job Name	Submitted By	Submitted	Started	Next Scheduled Run	Type	Cron Trigger ID
Manage Del Pause Job	Daily Inventory Sync	Trainer, Abdul	12/02/2025, 4:24 pm	13/02/2025, 12:00 am	14/02/2025, 12:00 am	Scheduled Apex	08edM00000Dh3p
Manage Del Pause Job	Daily Vehicle Order Processing	Trainer, Abdul	13/02/2025, 5:14 am	13/02/2025, 12:00 pm	14/02/2025, 12:00 pm	Scheduled Apex	08edM00000Dj99
Del	Metalytics Data Loader Job for Org: 00CDM00000JDX85	User, Integration	31/01/2025, 5:38 pm	12/02/2025, 7:42 pm	13/02/2025, 7:42 pm	Autonomous Data Loader Job	08edM00000CbmlO
	Program Milestone Computation Cron Job	Process, Automated	31/01/2025, 5:38 pm	13/02/2025, 6:30 am	13/02/2025, 1:29 pm	Program Milestone Computation Cron Job	08edM00000CbmlM
	Program Status Update Cron Job	Process, Automated	31/01/2025, 5:38 pm	13/02/2025, 9:31 am	13/02/2025, 6:30 pm	Program Status Update Cron Job	08edM00000CbmlN

Github Link:

<https://github.com/amaeshamalesh-dotcom/WhatNext-Vision-Motors-Shaping-the-Future-of-mobility-with-innovative-and-excellence.git>

Demo Link;

<https://drive.google.com/file/d/1z3PHUhSVmqiis9bNFMCW1ofd4FJyQkjr/view?usp=sharing>